ITSAR2 313 – SYSTEM INTEGRATION AND ARCHICTECTURE 2
LABORATORY REPORT: EXERCISE 1 MONOLITH VS MICROSERVICE

Aguinaldo, Emilio; Rizal, Jose
Information Technology, College of Computer Studies
Carlos Hilado Memorial State University

Abstract

This laboratory exercise explores the implementation and comparison of two fundamental software architectural patterns: Monolithic and Microservices architectures. A Student Information System was developed using both approaches to demonstrate the practical differences in design, implementation, scalability, and maintainability. Part A implements a single, tightly-coupled application with one database, while Part B decomposes the system into three independent services (Student, Course, and Enrollment) that communicate via HTTP REST APIs. The exercise provides hands-on experience in understanding when each architectural approach is appropriate for different business requirements and system scales.

I. INTRODUCTION

Theories and Concepts

Monolithic Architecture is a traditional software design approach where all components of an application are tightly coupled and deployed as a single unit. In this pattern, the user interface, business logic, and data access layers are interconnected within one codebase, sharing a single database. This architecture is characterized by simplicity in development and deployment but can become challenging to maintain and scale as the application grows.

Microservices Architecture is a modern approach that structures an application as a collection of loosely coupled, independently deployable services. Each service is responsible for a specific business capability, has its own database (database-per-service pattern), and communicates with other services through well-defined APIs, typically using HTTP/REST protocols.

Key Concepts Covered:

Single Responsibility Principle: Each microservice handles one domain (Students, Courses, Enrollments)
Service Independence: Services can be developed, deployed, and scaled independently
Inter-Service Communication: HTTP REST API calls between services
API Gateway Pattern: A single entry point that routes requests to appropriate microservices
Database Per Service: Each microservice maintains its own database to ensure loose coupling

Prerequisites:

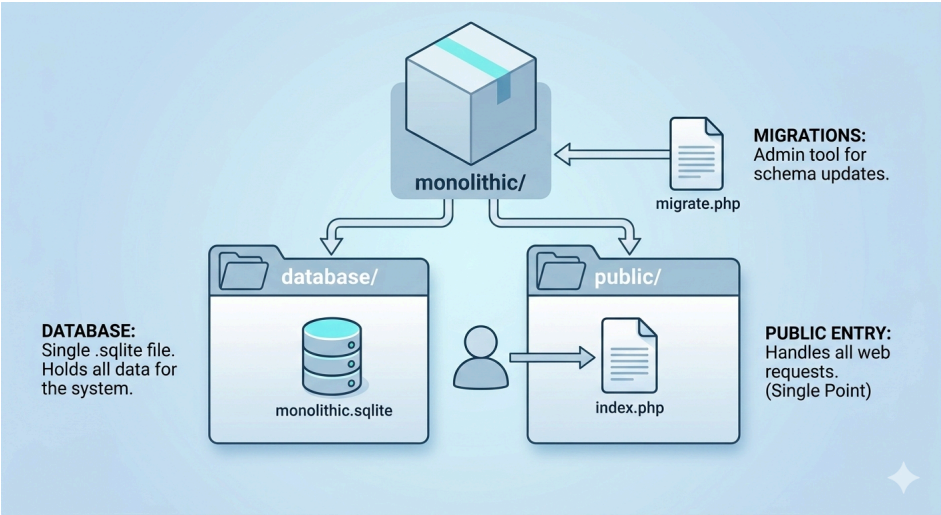
- PHP 8.2 or higher
- Understanding of RESTful API design
- Knowledge of HTTP methods (GET, POST, PUT, DELETE)

- SQLite database fundamentals
- Basic understanding of client-server architecture

II. METHODOLOGY / IMPLEMENTATION

The implementation process followed a systematic approach to build a Student Information System using two distinct architectural patterns. The objective was to demonstrate the practical differences between Monolithic and Microservices architectures while achieving full CRUD functionality for managing students, courses, and enrollments.

Part A — Monolithic System Implementation
Step 1: Project Structure



Step 2: Database Design
A single SQLite database (monolithic.sqlite) was created containing three tables:

students - Stores student information (id, student_number, name, email, phone, address, etc.)
courses - Stores course information (id, course_code, name, description, credits, etc.)
enrollments - Links students to courses with foreign key relationships

Step 3: API Implementation
All endpoints were implemented in a single index.php file:

Method	Endpoint	Description
POST	/students	Create a new student
GET	/courses	List all courses
POST	/enrollments	Create a new enrollment

GET	/enrollments/{id}	Get enrollment by ID

Step 4: Running the Application

```
php monolithic/migrate.php          Run migrations
php -S localhost:8080 -t monolithic/public   Start server
```

Part B — Microservices System Implementation

Step 1: Service Decomposition
The system was split into three independent services:

Service	Port	Database	Responsibility
Student Service	8001	student_service.sqlite	CRUD operations for students
Course Service	8002	course_service.sqlite	CRUD operations for courses
Enrollment Service	8003	enrollment_service.sqlite	Enrollment management, grades
API Gateway	8000	-	Request routing, authentication

Step 2: Inter-Service Communication

The API Gateway uses ServiceClient.php for HTTP communication with microservices
Enrollment Service validates student/course existence by calling respective services
Data enrichment: Enrollment responses include student names and course details fetched via HTTP

Step 3: Database Per Service
Each service maintains its own SQLite database, ensuring:

- Loose coupling between services
- Independent data management
- Service isolation

Step 4: Running the Services

```
start-services.bat     Start all microservices (ports 8001-8003)
start-gateway.bat      Start API Gateway (port 8000)
```

III. EXPERIMENTAL FINDINGS / OBSERVATIONS

Part A — Monolithic System Testing
Upon starting the monolithic application on port 8080, the initial health check endpoint was tested to verify the system status. The response confirmed that the service was running correctly and displayed comprehensive statistics showing that the database contained 5 pre-seeded students, 8 courses, and 10 enrollment records. This immediate visibility into the entire system's state demonstrated one of the key characteristics of monolithic architecture — the ability to query and monitor all data from a single point.

The student creation endpoint was tested by submitting a POST request with a JSON payload containing a name and email address. The system successfully processed the request, automatically generating a unique student number (STU-00006) following the established pattern, and returned the complete student record with all fields including timestamps. The validation logic correctly rejected duplicate email addresses, returning a 422 status code with an appropriate error message. This behavior confirmed that data integrity constraints were properly enforced at the database level through the UNIQUE constraint on the email column.

Testing the course listing endpoint returned all 8 courses in a single JSON array, complete with course codes, descriptions, credit units, department assignments, and semester information. The response time was notably fast since the query executed directly against the local SQLite database without any network overhead. The data was retrieved using a simple SQL SELECT statement, demonstrating the straightforward nature of data access in a monolithic system where all tables reside in the same database.

The enrollment creation process was tested by attempting to enroll an existing student in a course. The system correctly validated that both the student_id and course_id referenced existing records before creating the enrollment. When attempting to create a duplicate enrollment (same student and course combination), the system returned an error message indicating that the student was already enrolled in the specified course. The enrollment retrieval endpoint successfully returned enriched data that included not only the enrollment details but also the student's name, student number, email, course code, and course name — all fetched through SQL JOIN operations in a single database query.

Part B — Microservices System Testing
The microservices implementation required starting four separate processes: the Student Service on port 8001, Course Service on port 8002, Enrollment Service on port 8003, and the API Gateway on port 8000. Each service was observed to start independently, and the health check endpoints for each service confirmed their individual operational status. A significant observation was that stopping one service (for example, the Course Service) did not affect the continued operation of the other services, demonstrating the fault isolation characteristic of microservices architecture.

When testing the Student Service directly, the CRUD operations functioned identically to the monolithic version in terms of functionality. However, the service only had access to student data stored in its dedicated student_service.sqlite database. Attempts to query course or enrollment information from this service returned 404 errors, confirming that each service was properly isolated and responsible only for its designated domain.

The inter-service communication was most evident when testing the Enrollment Service. Upon receiving a request to create a new enrollment, the service initiated HTTP calls to both the Student Service and Course Service to validate that the referenced student and course actually existed. This validation process introduced observable latency compared to the monolithic version, as each validation required a network round-trip. The console logs showed the sequence of HTTP requests being made, confirming the distributed nature of the operation.

Data enrichment in the microservices architecture required additional HTTP calls. When retrieving an enrollment record, the Enrollment Service stored only the student_id and course_id in its local database. To return a complete response with student names and course details, the service made subsequent calls to the Student and Course services to fetch the supplementary information. This process was noticeably slower than the monolithic version's single JOIN query but provided the benefit of real-time data consistency — if a student's name was updated in the Student Service, the enrollment response would immediately reflect that change without requiring any synchronization.

The API Gateway successfully aggregated responses from multiple services. The gateway's health check endpoint demonstrated this by collecting status information from all three microservices and presenting a unified view of the entire system's health. When one microservice was intentionally stopped, the gateway's health check accurately reported that service as unavailable while showing the others as operational. This centralized monitoring capability proved valuable for system administration and debugging.

An interesting observation occurred when testing concurrent requests. The monolithic system handled simultaneous requests sequentially due to PHP's synchronous nature, while the microservices could theoretically process requests in parallel across different services. However, for operations requiring inter-service communication (like enrollment creation), the sequential HTTP calls created a bottleneck that offset some of the parallelization benefits.

IV. LAB DISCUSSIONS / COMPUTATIONS

Architectural Comparison

Aspect	Part A (Monolithic)	Part B (Microservices)
Applications	1	4 (Gateway + 3 services)
Databases	1	4 (1 per service)
Communication	Direct function calls	HTTP REST API
Scaling	Scale entire app	Scale individual services
Deployment	Single deployment	Independent deployments
Complexity	Simple	More complex
Fault Isolation	Single point of failure	Isolated failures

Performance Considerations

Monolithic Advantages:

- Lower latency (no network calls between components)
- Simpler debugging and tracing
- Easier transaction management
- Less operational overhead

Microservices Advantages:

- Independent scaling of high-load services
- Technology flexibility per service
- Fault isolation (one service failure doesn't crash others)
- Easier team organization around services
- When to Use Each Architecture

Use Monolithic When:

- Small team or early-stage project
- Simple domain with limited complexity
- Tight deadline requirements
- Limited operational expertise

Use Microservices When:

-
- Large, complex domains with clear boundaries
- Need for independent scaling
- Multiple teams working on different features
- High availability requirements

V. CONCLUSIONS

This laboratory exercise successfully demonstrated the implementation of both Monolithic and Microservices architectures for a Student Information System. The following conclusions were derived:

Monolithic architecture provides simplicity and ease of development for smaller applications. With a single codebase and database, it offers straightforward debugging, lower latency, and simpler deployment processes. It is ideal for projects with limited scope or teams starting new applications.

Microservices architecture offers superior scalability, fault isolation, and flexibility at the cost of increased complexity. The decomposition into Student, Course, and Enrollment services demonstrated how business capabilities can be independently developed, deployed, and scaled.

Inter-service communication via HTTP REST APIs introduces network latency but enables loose coupling. The Enrollment Service's ability to validate data across services showcased the power of distributed systems.

Database-per-service pattern ensures service autonomy but requires careful consideration of data consistency and cross-service queries.

The choice between architectures should be based on project requirements, team size, scalability needs, and operational capabilities rather than following trends.

The exercise achieved its objectives by providing practical experience in implementing both patterns, understanding their trade-offs, and recognizing appropriate use cases for each architectural approach.

References

Fowler, M. (2014). Microservices: A Definition of This New Architectural Term. martinfowler.com

Newman, S. (2021). Building Microservices: Designing Fine-Grained Systems (2nd ed.). O'Reilly Media.

Richardson, C. (2018). Microservices Patterns: With Examples in Java. Manning Publications.

PHP Documentation. (2024). Built-in Web Server. php.net/manual/en/features.commandline.webserver.php

SQLite Documentation. (2024). SQLite Home Page. sqlite.org

Laravel Documentation. (2024). HTTP Client. laravel.com/docs/http-client

RESTful API Design Best Practices. REST API Tutorial. restfulapi.net